RETAILPULSE

AI-Powered Customer Analytics & Demand Forecasting Platform

TURN RETAIL TRANSACTIONS INTO ACTIONABLE BUSINESS INSIGHTS

PROJECT DOCUMENTATION

Submitted by: Shekhar Shelke

Internship Domain: Data Science & Analytics

Organization: Zidio Development

Date: 03 November 2026

1. Project Overview

RetailPulse is a retail analytics application that organizes transaction data into sales insights, customer segments, churn-risk estimates, monthly sales forecasts and historical-demand-based reorder suggestions.

Vision

To provide a single, accessible analytics workspace where retail stakeholders can explore business performance and use data-driven indicators to support customer and inventory decisions.

Objectives

- Prepare and validate retail transaction data for analysis.
- Explore sales trends by month, product and country.
- Segment customers using Recency, Frequency and Monetary (RFM) measures.
- Estimate customer churn risk using a Random Forest classification workflow.
- Generate a baseline monthly sales forecast using a Random Forest regression workflow.
- Calculate indicative reorder thresholds from historical product demand.
- Present analytical outputs through a multi-page Streamlit dashboard.

Target users and use cases

Target user	Example use case
Retail / business analyst	Review sales patterns and compare products or countries.
Customer relationship team	Inspect RFM groups and prioritize customers for retention campaigns.
Inventory planner	Review high-demand items and indicative reorder thresholds.
Business stakeholder	Use dashboard summaries to support planning discussions.

Business value

The project reduces the effort required to inspect multiple prepared datasets manually, brings several analytical views into one interface, and provides repeatable baseline calculations. It is a decision-support prototype; its outputs should be checked against operational context before business action.

Non-functional goals and current status

Quality area	Proposed target / expectation	Evidence status
Usability	Clear navigation and readable charts across 12 dashboard pages.	Implemented locally; user testing not documented.
Performance	Dashboard pages should load smoothly on the intended dataset.	No measured latency benchmark supplied.
Reliability	Handle missing/invalid fields with clear errors or cleaning steps.	Some preprocessing is implemented; full test coverage not verified.
Scalability	Separate processing modules from dashboard presentation.	Modular source layout exists; load testing not done.
Model quality	Report task-appropriate metrics on a sufficiently large holdout.	Forecast MAE/RMSE available; churn metrics need rerun evidence.

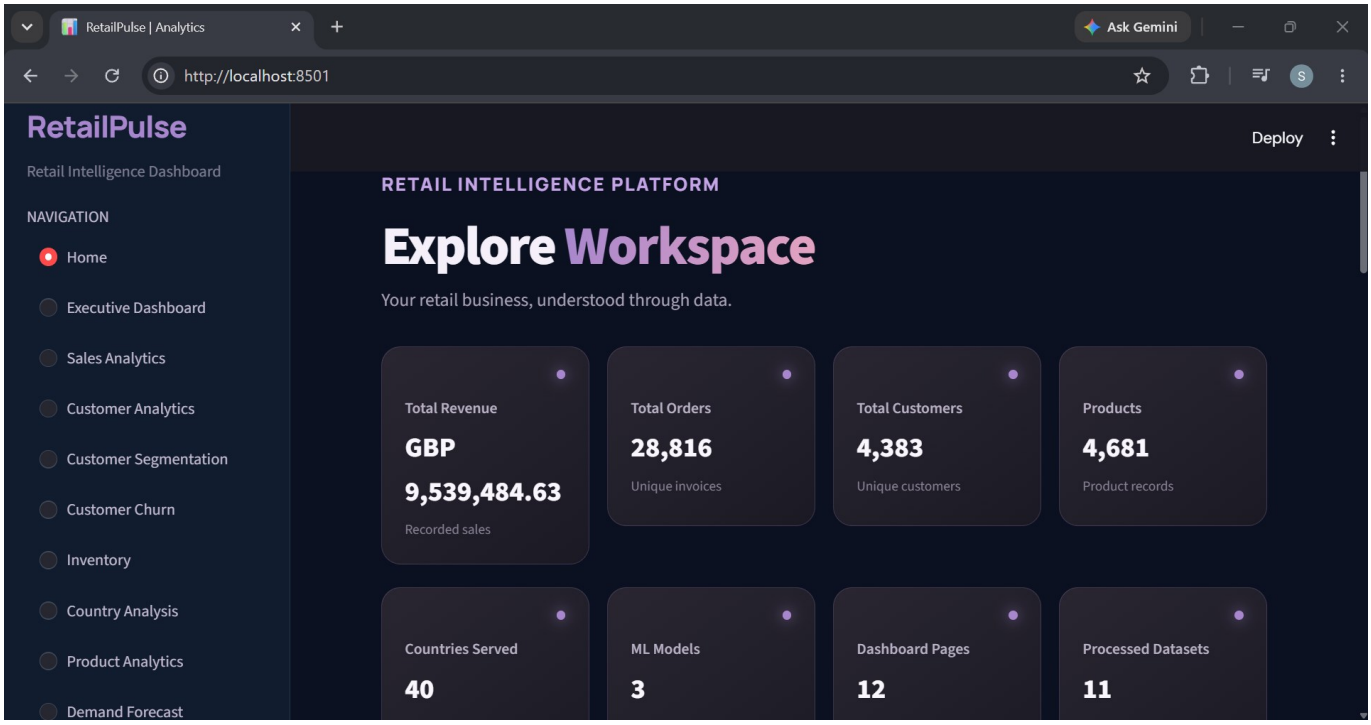
2. Key Features

RetailPulse provides key features for sales analytics, customer segmentation, churn prediction, demand forecasting, and inventory recommendations. These features help users explore retail data and support data-driven business decisions.

ID	Feature	Description	Acceptance criteria
F01	Sales analytics	Date/country filters, sales KPIs, monthly trend, top products/countries.	Filters update the displayed metrics/charts; filtered data can be exported.
F02	Executive dashboard	Consolidated overview of key retail indicators.	Summary values match the prepared source datasets.
F03	Customer analytics	Customer-level RFM information and spending views.	Customer table and summary charts load from RFM output.
F04	RFM segmentation	Groups customers into five rule-based segments.	Segment totals reconcile to 4,314 records in supplied output.
F05	Churn risk	Random Forest classification and risk probability output.	Prediction file includes Churn, probability and risk category fields.
F06	Demand forecast	Monthly sales forecast and actual-vs-predicted comparison.	Forecast output contains date and predicted sales values.
F07	Inventory suggestions	Historical average demand converted to reorder thresholds.	Output contains reorder level, demand rank and category.
F08	Country/product analysis	Summaries of sales by geography and product.	Top-country and top-product summaries are visible.
F09	CSV data explorer	Inspect prepared CSV datasets.	Selected dataset can be viewed without application errors.
F10	Export	Download filtered/derived datasets where supported.	Downloaded CSV opens and includes expected columns.

Dashboard navigation map

Home → Executive Dashboard → Sales Analytics → Customer Analytics → Customer Segmentation → Customer Churn → Inventory → Country Analysis → Product Analytics → Demand Forecast → Business Insights → CSV Data Explorer.



3. Technology Stack

The following table describes the technologies and tools used in the development of the RetailPulse – AI-Powered Customer Analytics & Demand Forecasting Platform.

Category	Technology	Rationale / alternatives
Language	Python	Main language for data preparation and ML; alternatives include R or Julia.
Data manipulation	Pandas, NumPy	Tabular cleaning, aggregation and numerical operations; alternatives include Polars.
Machine learning	scikit-learn	Used for Random Forest classification and regression, along with data preprocessing.
Dashboard	Streamlit	Python-native interactive dashboard; alternatives include Dash or FastAPI + frontend.
Visualization	Plotly, Matplotlib, Seaborn	Interactive and static charts; alternatives include Altair or Power BI.
Storage / exchange	CSV and Excel files	Simple project data interchange; alternatives include PostgreSQL or cloud object storage.
Development	Git / GitHub (repository workflow)	Version control and source hosting;
Deployment	Local Streamlit run verified in project instructions	The application was tested and run locally using Streamlit. Public cloud deployment has not yet been completed.

Project structure

- dashboard/app.py — Streamlit application entry point.
- src/preprocessing/ — data cleaning, EDA and feature engineering.
- src/segmentation/ — RFM customer segmentation.
- src/churn/ — churn prediction workflow.
- src/forecasting/ — monthly sales forecasting.
- src/inventory/ — historical-demand reorder calculations.
- data/ — source and generated CSV datasets.
- docs/ and scripts/ — documentation and utility scripts.

4. Docker Containerization and Deployment

The RetailPulse application was successfully containerized using Docker to provide a consistent and portable execution environment.

Docker Configuration

Docker Image: retailpulse-app:latest

- **Container Name: retailpulse-container**
- **Application Port: 8501**
- **Local Access URL: http://localhost:8501**

Implementation Steps

1. **Created a Dockerfile to define the application environment.**
2. **Installed the required Python dependencies inside the Docker image.**
3. **Built the Docker image using the Docker CLI.**
4. **Started the application inside a Docker container.**
5. **Accessed and tested the Streamlit dashboard through the local browser.**

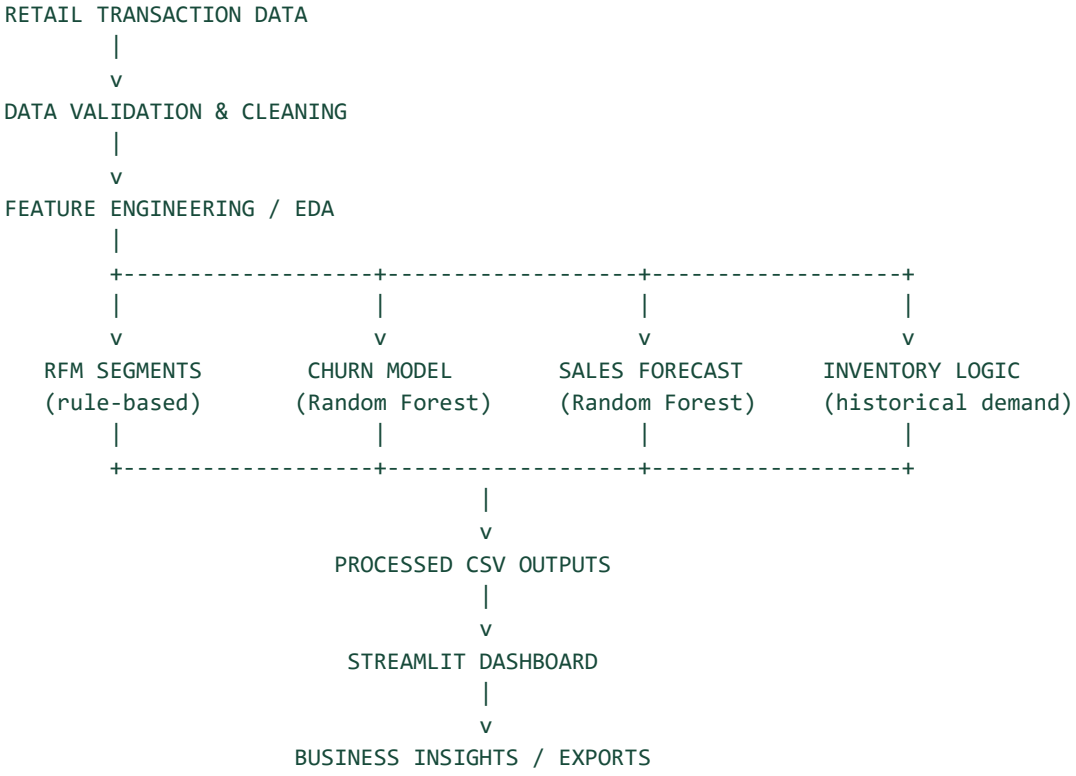
Testing Results

The Docker image was built successfully, and the container started without errors. The RetailPulse dashboard was accessible at port 8501, and its pages and visualizations were tested successfully.

Result: Docker containerization and local deployment testing completed successfully.

5. Architecture Overview

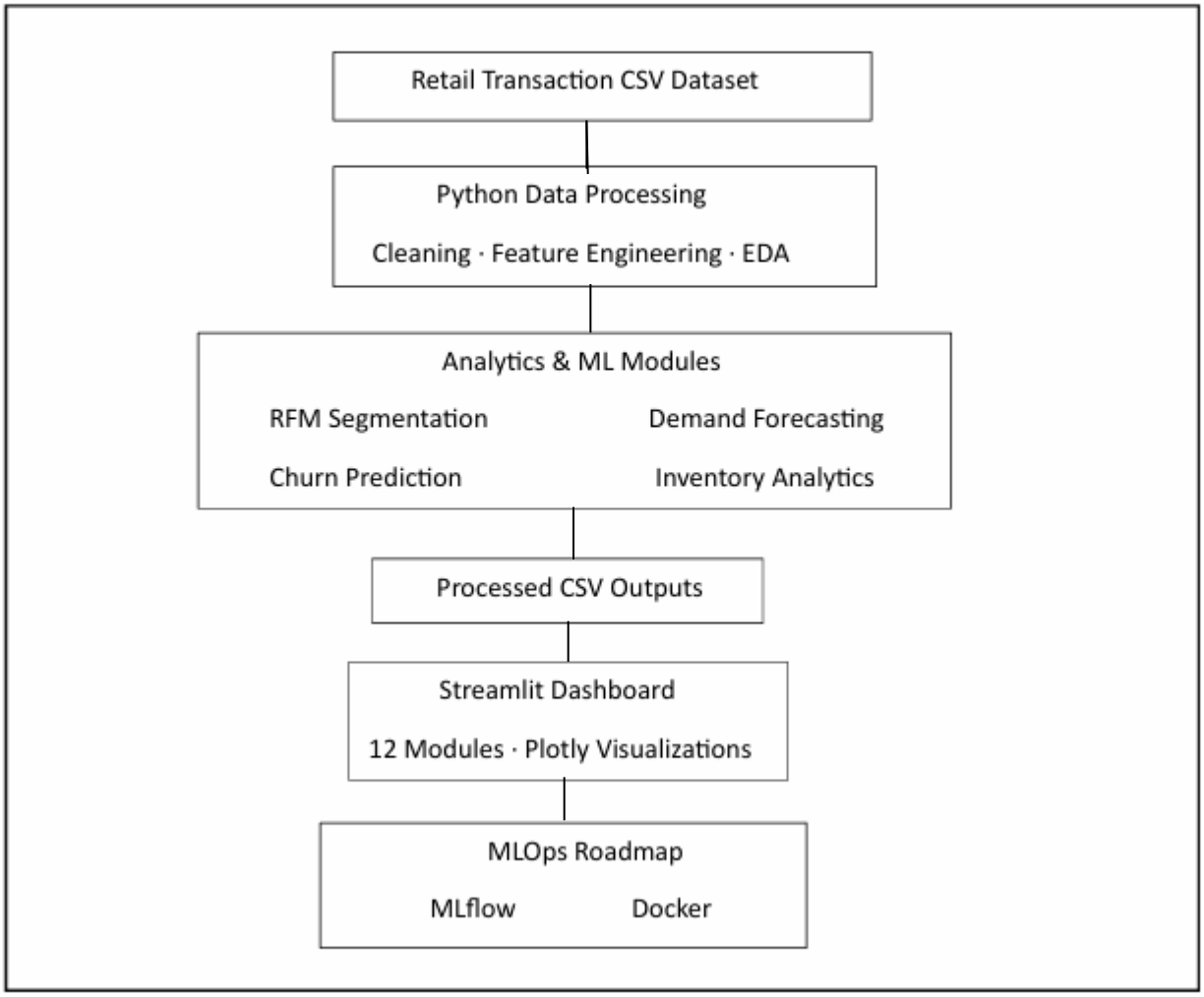
RetailPulse follows a modular, local data processing architecture. The system takes retail transaction data as input, processes and transforms it using Python, performs customer and business analytics, and presents the generated insights through an interactive Streamlit dashboard.



Data flow and MLOps boundary

The implemented workflow is primarily batch/offline: scripts read CSV files, create processed outputs and the dashboard loads those outputs. The supplied evidence does not verify automated scheduling, CI/CD, model registry, drift detection, continuous retraining, Docker deployment or Kubernetes orchestration. Treat these as roadmap items until configured and tested.

Actual Data Flow + MLOps Boundary



6. Detailed Execution Timeline

The RetailPulse project was developed through a series of structured phases, starting with dataset inspection and progressing towards data processing, analytical model development, dashboard integration, and documentation.

Phase	Work performed / to verify	Deliverable / checkpoint	Actual dates
1. Project setup	Repository structure, environment and dependencies.	Runnable local project; requirements file.	05–06 October 2026
2. Data inspection	Inspect source files, schemas, missing values and date ranges.	Data inventory and cleaning plan.	07–10 October 2026
3. Preprocessing & EDA	Clean transaction data, derive fields and generate summaries/plots.	Prepared sales data and EDA outputs.	11–14 October 2026
4. Customer analytics	Calculate RFM scores and assign rule-based segments.	customer_rfm_segments.csv and segment charts.	15–17 October 2026
5. Churn workflow	Build 90-day target, customer features and Random Forest classifier.	customer_churn_predictions.csv.	18–20 October 2026
6. Forecast workflow	Aggregate monthly sales, create lag features and train baseline model.	Forecast test results and next-period forecast.	21–23 October 2026
7. Inventory workflow	Aggregate product demand and calculate reorder thresholds.	inventory_recommendations.csv.	24–25 October 2026
8. Dashboard integration	Build pages and connect prepared datasets to charts/tables.	12-page Streamlit dashboard.	26–29 October 2026
9. Validation & packaging	Run smoke checks, capture screenshots and update documentation.	Final verified report, README and submission package.	30 October–01 November 2026

Milestone checkpoints

- M1 — Source data and project environment validated.
- M2 — Core analytical output files generated.
- M3 — Dashboard pages connected to outputs.
- M4 — End-to-end local run and screenshot evidence completed.
- M5 — Final repository and submission artifacts reviewed.

6. Technical Highlights

RetailPulse combines data preprocessing, customer analytics, machine learning, demand forecasting, inventory analysis, and interactive dashboard visualization in a single Python-based application.

Feature engineering

Workflow	Features / transformations
Sales	InvoiceDate parsing; TotalSales and calendar fields including Year, Month, Day, DayOfWeek and YearMonth.
RFM	Recency = days since last purchase; Frequency = distinct invoice count; Monetary = sum of TotalSales. Scores use ranked quantile bins.
Churn	Recency, Frequency, Monetary and TotalQuantity; target = no purchase in the 90 days after the historical cutoff.
Monthly Sales Forecasting	Monthly TotalSales aggregation; Month, Year, TimeIndex and Lag1–Lag3 features.
Inventory	UnitsSold, Revenue, distinct invoice count, observed date span and AverageDailyDemand.

Model and analytical results

Component	Implementation / result	Interpretation
RFM segmentation	4,314 customers: Others 1,367; Champions 1,085; Loyal 804; At Risk 699; Recent 359.	Rule-based segmentation, not clustering.
Churn classifier	RandomForestClassifier, 200 trees, class_weight=balanced; 3,379 output rows. Churn=0: 1,942; Churn=1: 1,437.	Accuracy/AUC/F1 not verified in saved evidence; rerun and record classification report.
Sales forecast	RandomForestRegressor, 200 trees; 13 monthly points → 10 lagged rows → 7 train / 3 test. MAE ≈ 477,198.11; RMSE ≈ 499,340.66.	Only three test months; treat as a baseline with substantial uncertainty.
Inventory	5,532 product rows; reorder level = ceil(average daily demand × assumed 10 days).	Historical-demand threshold, not live-stock replenishment.

MLOps practices: implemented vs planned

Practice	Status supported by archive
Modular Python source	Implemented – Separate Python modules are available for preprocessing, segmentation, churn, forecasting, and inventory.
Fixed random seeds	Random state 42 in inspected Random Forest scripts.
Data/model version registry	Not verified.
Automated retraining pipeline	Not verified.
Drift detection / monitoring	Not verified.
CI/CD and production deployment	Not verified.
Docker / Kubernetes / MLflow	Planned – Project folders exist

Challenges Faced and Solutions

During the development of RetailPulse, handling and preparing retail transaction data was an important challenge. Data preprocessing using Python and Pandas helped organize the data for analysis. I also checked the generated outputs and dashboard results to identify inconsistencies.

7. Deployment & Operations

RetailPulse currently runs as a local Streamlit application. The dashboard uses processed CSV files generated by the project's data-processing scripts. Public deployment and production monitoring have not yet been verified.

Area	Current verified description	Final submission detail
Platform	Deployment Status: Local deployment completed. Hosting Platform: Local Streamlit application. Public URL: https://retailpulse-analytics-171.streamlit.app/	Hosting provider / URL:- Streamlit Community Cloud
Entry point	dashboard/app.py	Launch command: streamlit run dashboard/app.py
Dependencies	requirements.txt	Python version and package versions: requirements.txt
Docker Image	retailpulse-app:latest	Docker image built successfully,
Docker Container	retailpulse-container	container started successfully
Docker Port	8501	
Docker Deployment Status	Successfully built and tested locally	Streamlit dashboard was tested locally at http://localhost:8501
Kubernetes Deployment Name	retailpulse-deployment	RetailPulse – AI-Powered Customer Analytics and Demand Forecasting Platform
Configuration File	k8s/deployment.yaml	local Docker Desktop Kubernetes environment.
Access Command:	kubectl port-forward service/retailpulse-service 8502:8501	verified.
Local UR	http://localhost:8502	Kubernetes deployment was successfully tested
CI/CD	No working pipeline verified.	Workflow URL / status, if implemented: NA
Monitoring	No drift or production monitoring verified.	Tool, metric, alert evidence: NA
Security	Do not include secrets or credentials in repository.	Secret scan / access check: NA
Operations	Batch scripts create CSV outputs used by dashboard.	Refresh procedure and owner: CSV outputs consumed by the Streamlit dashboard

Pre-release validation checklist

Run installation from a clean environment. ☐

☐ Run preprocessing and each analytical script.

☐ Check output file names, columns, row counts and missing values.

☐ Launch the Streamlit dashboard and open all 12 pages.

☐ Compare key dashboard values with source CSV calculations.

☐ Test CSV downloads and browser refresh behavior.

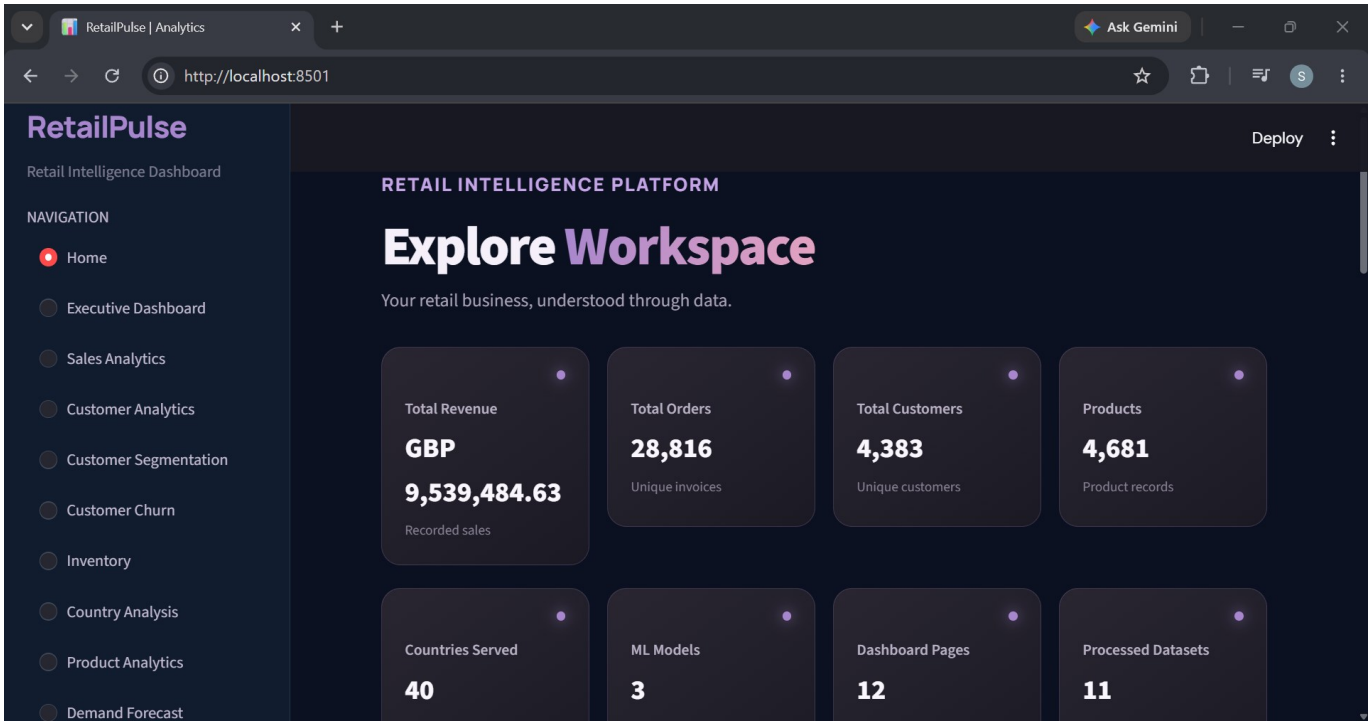
☐ If deployed, test public HTTPS access without login.

☐ Capture actual screenshots and record the build date.

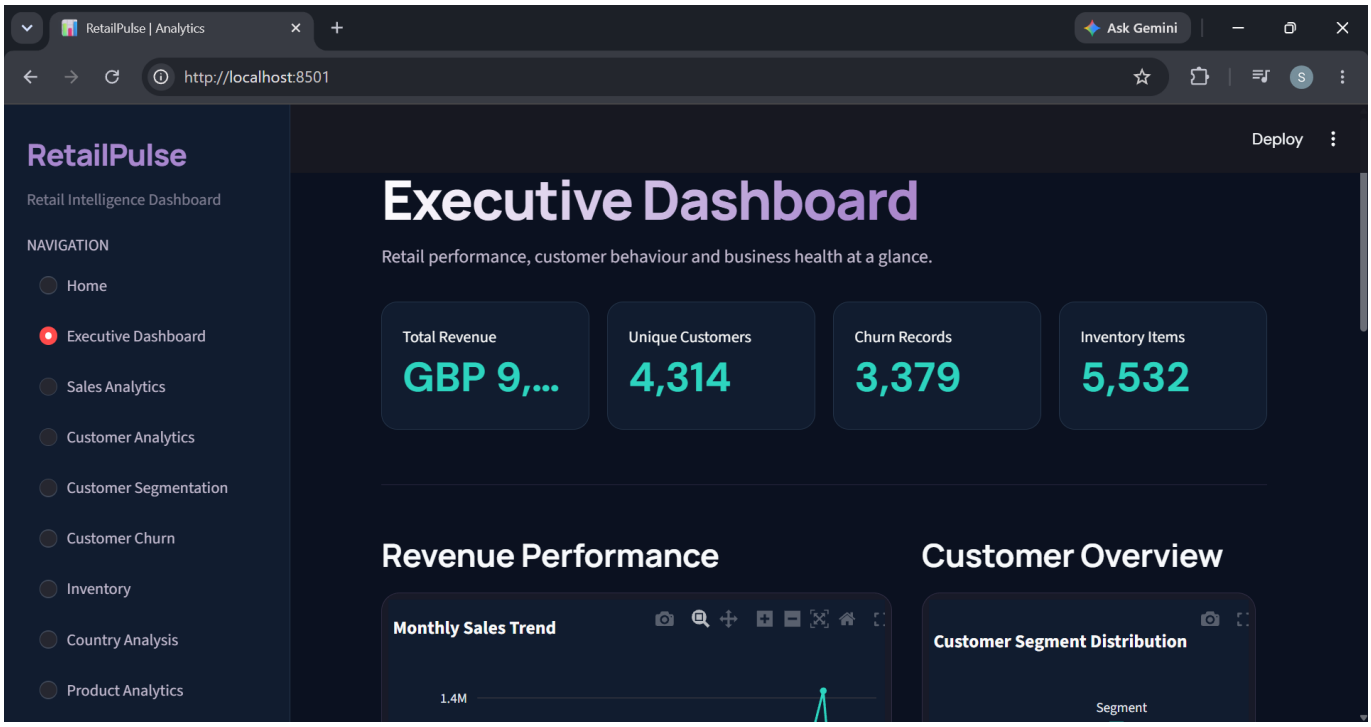
8. Visual Evidence — Module-wise Screenshot Slots

.

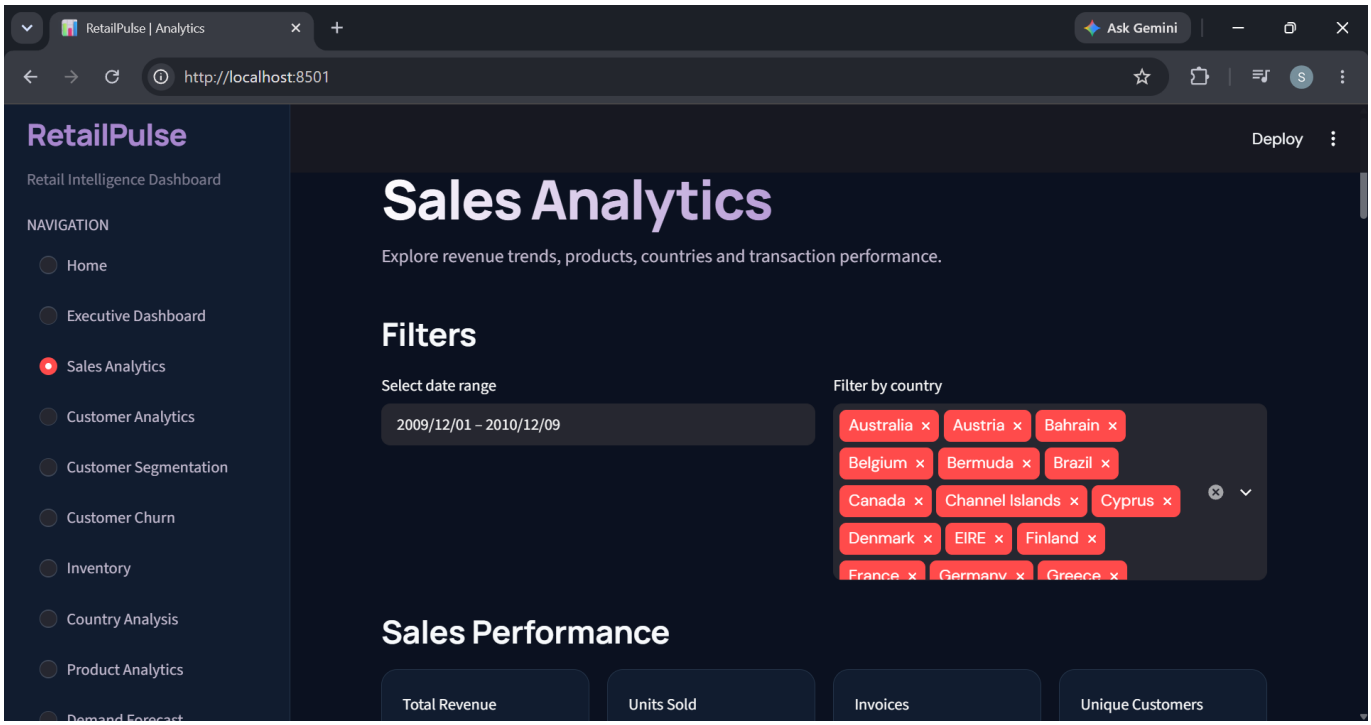
1.Home



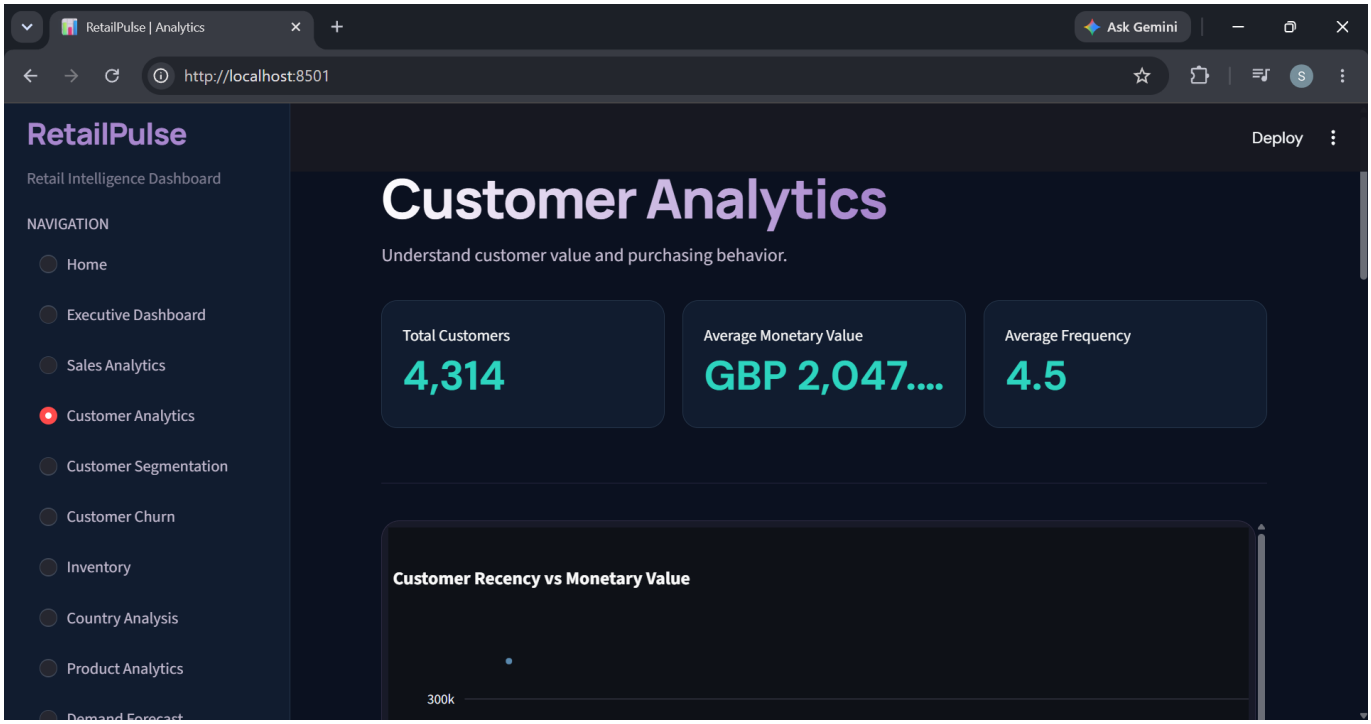
2.Executive Dashboard



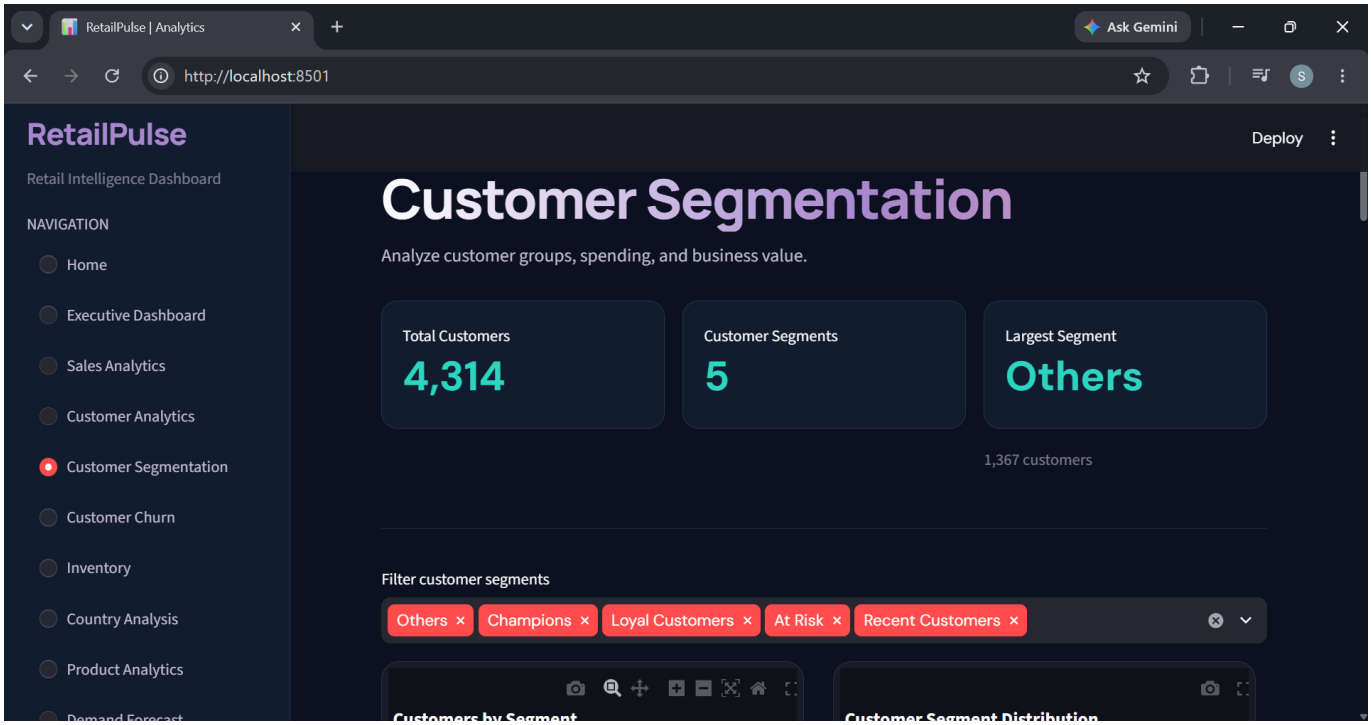
3.Sales Analytics



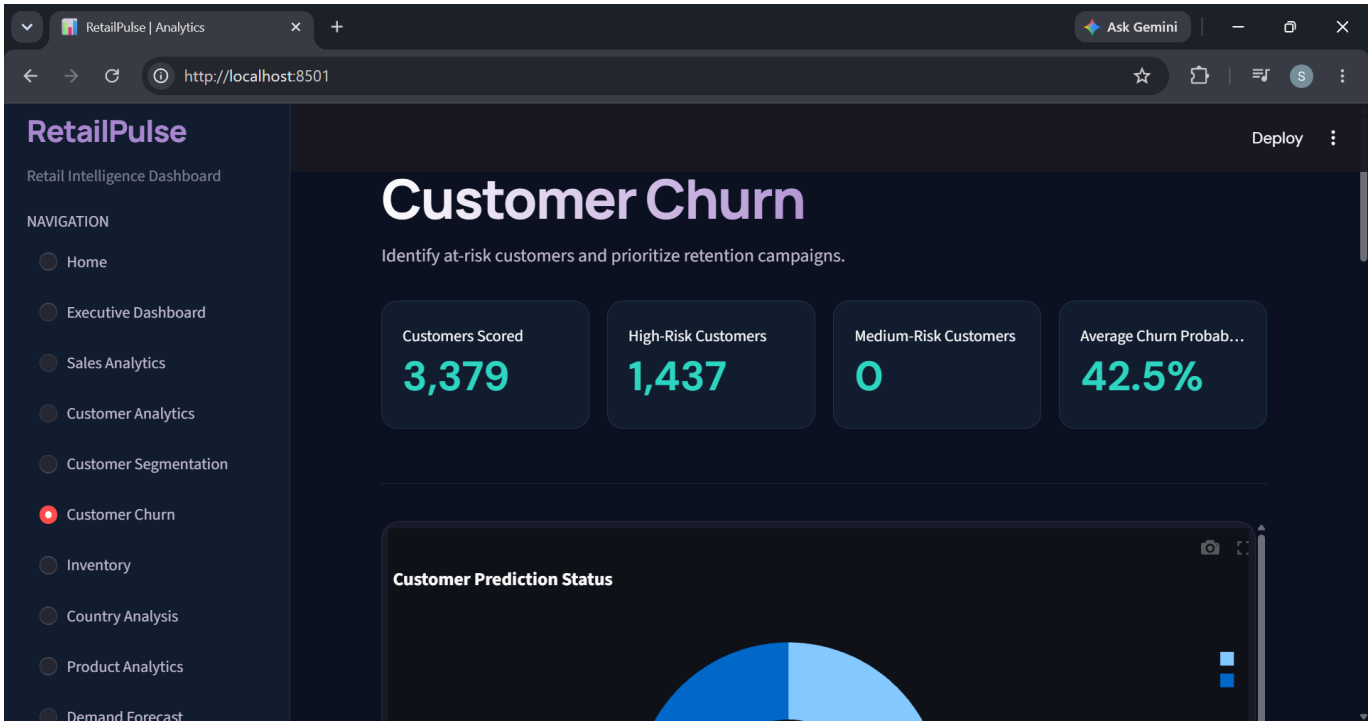
4.Customer Analytics



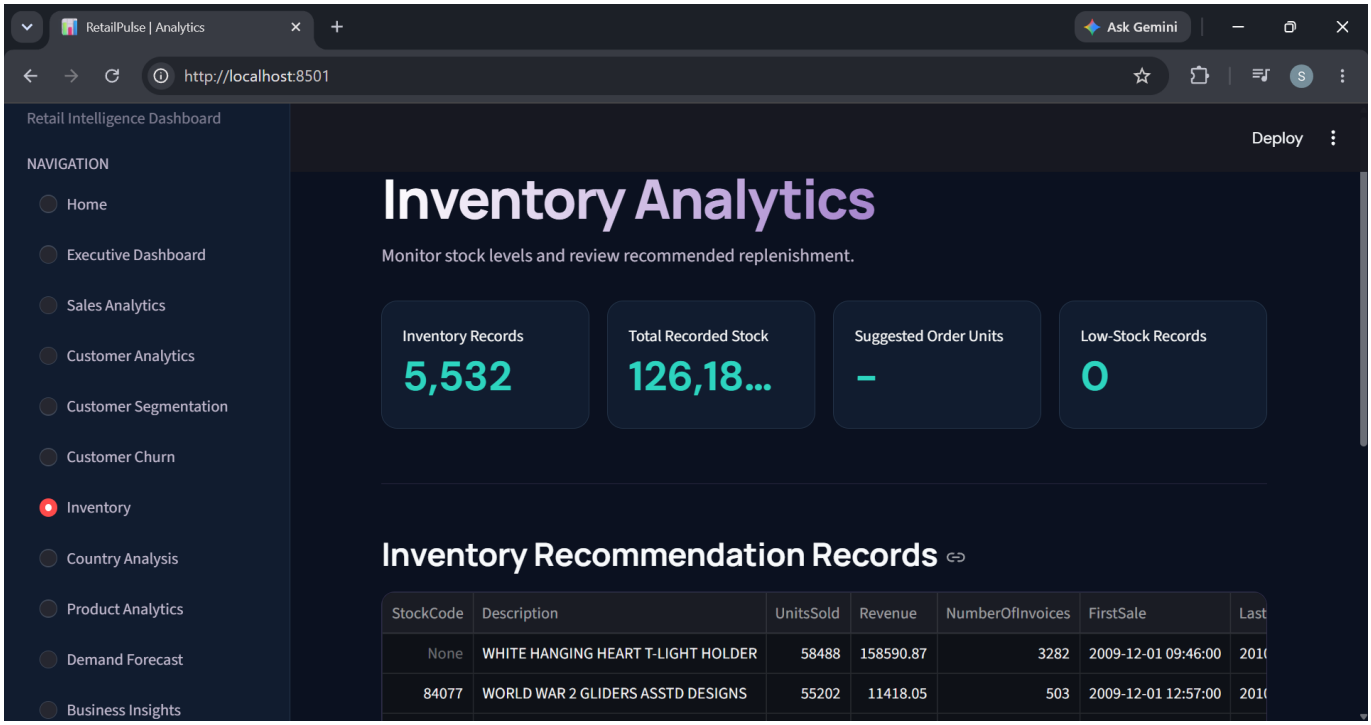
5.Customer Segmentation



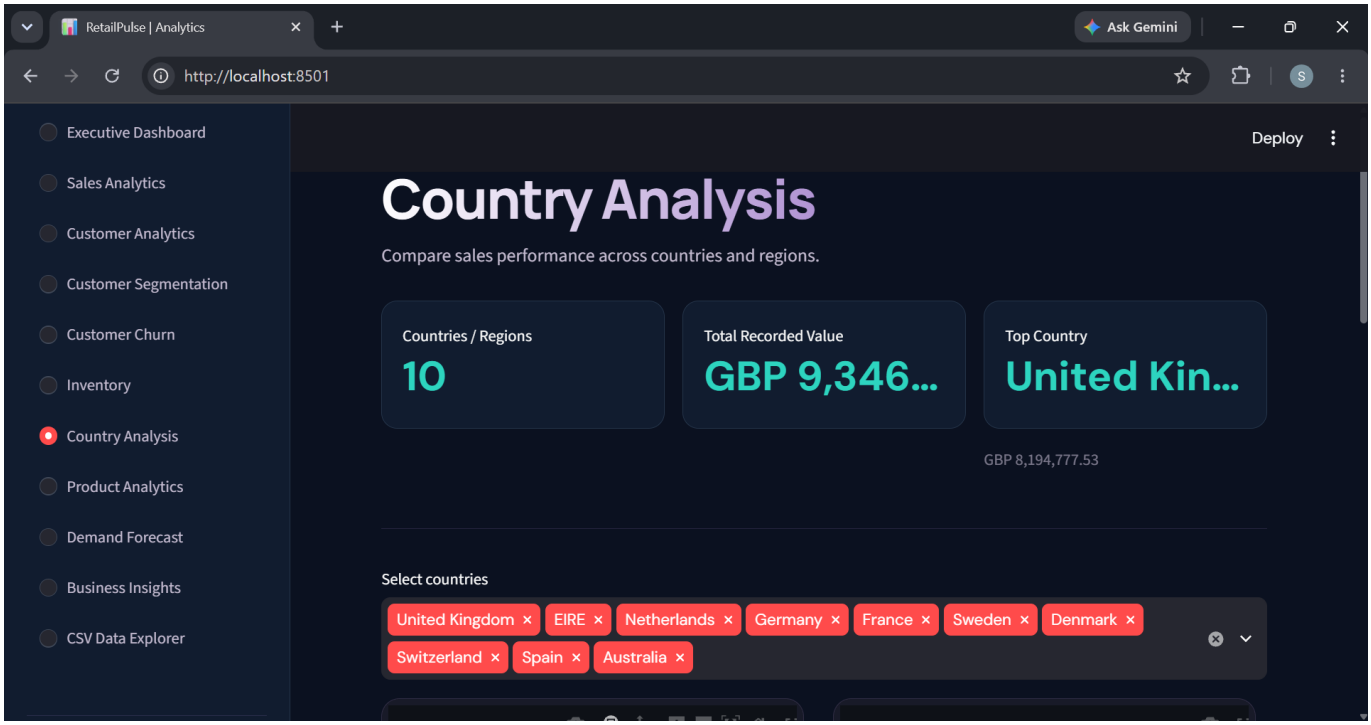
6.Customer Churn



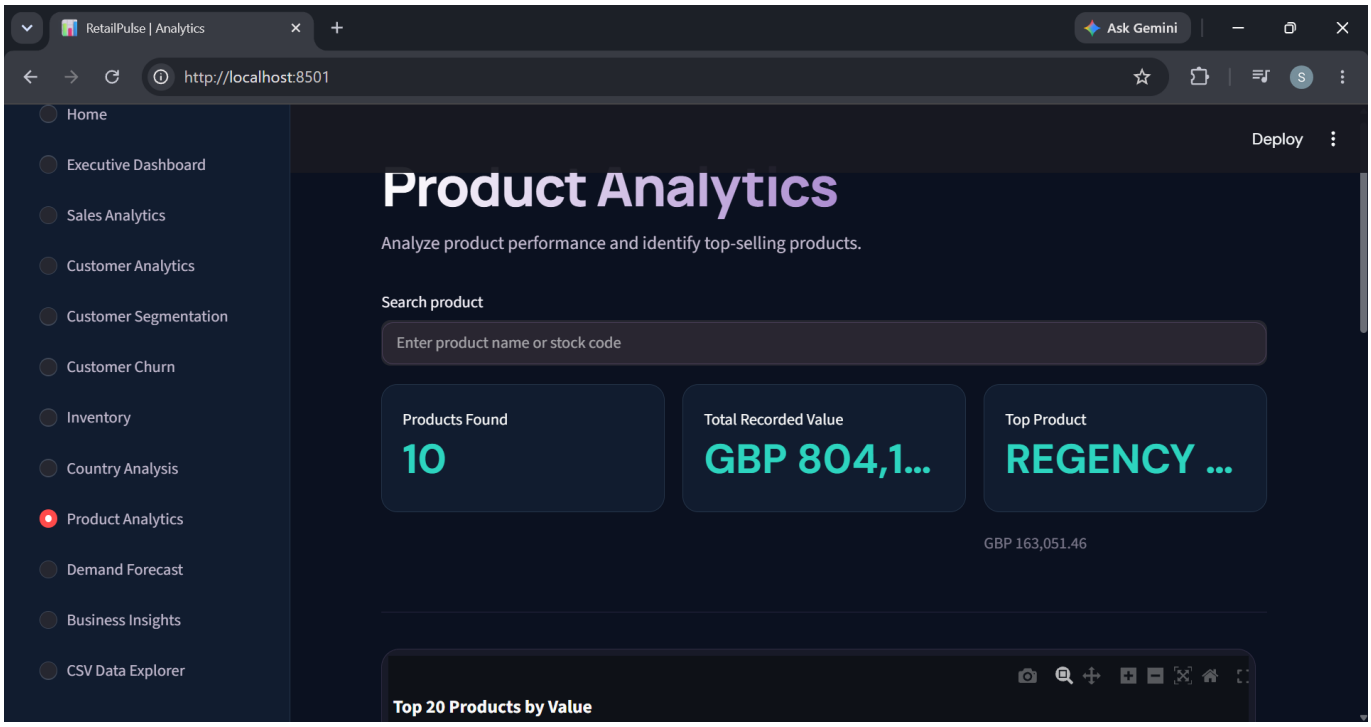
7.Inventory Analytics



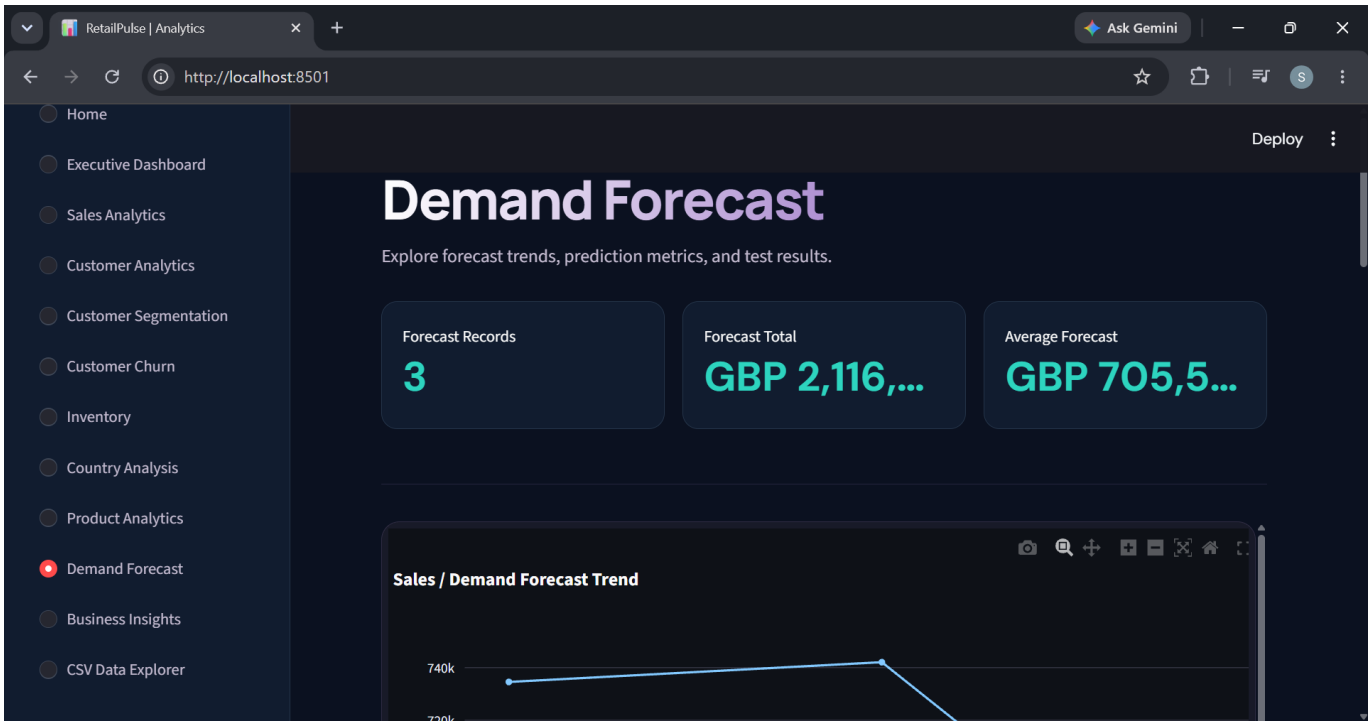
8.Country Analysis



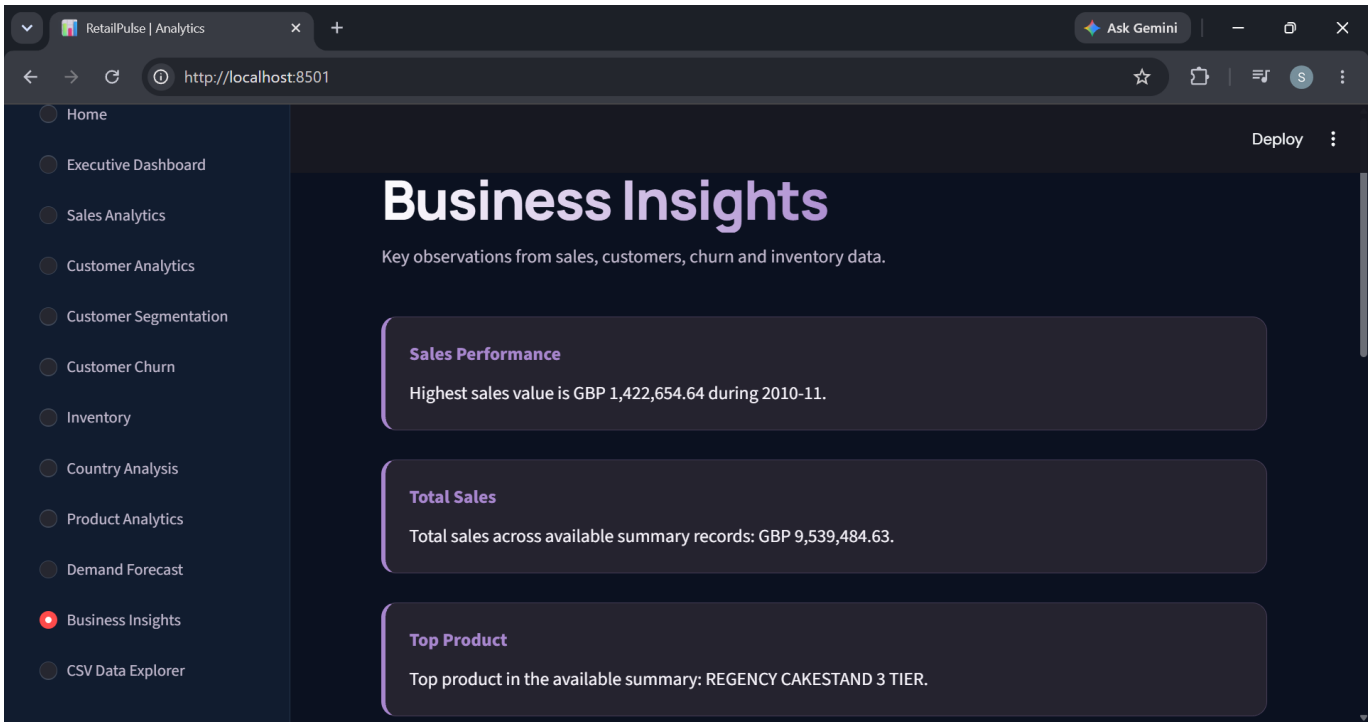
9.Product Analytics



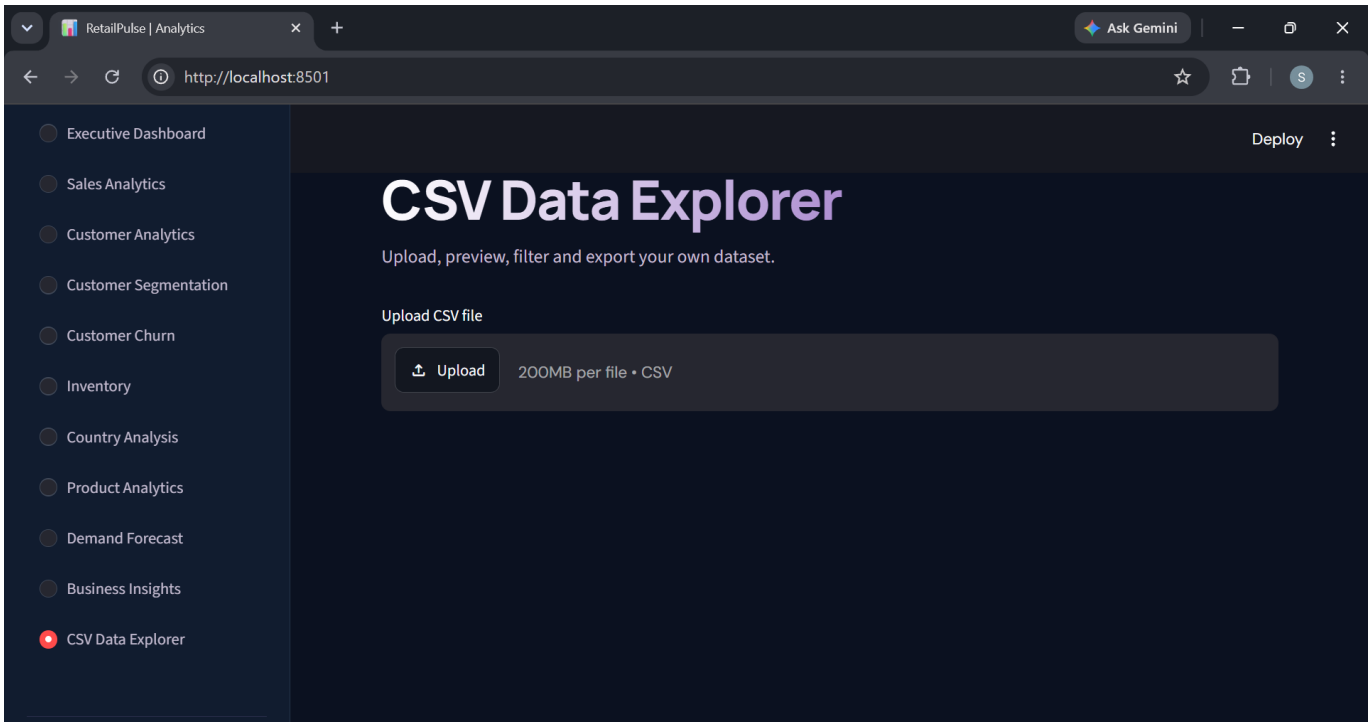
10.Demand Forecast



11. Business Insights



12. CSV Data Explorer



9. Personal Reflection

Write this section in your own words. The prompts are provided to help you describe your actual contribution and learning.

9.1 Key learnings

During this project, I learned how to work with real-world retail transaction data. I gained practical experience in data cleaning, exploratory data analysis, feature engineering, customer segmentation, machine learning model training, and dashboard development.

I learned how to handle missing values, remove invalid records, calculate customer RFM metrics, and build Random Forest models for churn prediction and demand forecasting.

Example: I used RFM analysis to divide customers into groups such as Champions, Loyal Customers, At Risk, and Recent Customers.

9.2 Industry practices applied

During the development of RetailPulse, I followed several software development practices:

- Modular code structure for different project components.
- Separate folders for data, source code, models, and dashboard.
- Data preprocessing and validation before analysis.
- Documentation of system architecture and project modules.
- Separation of data processing logic and dashboard interface.
- Use of GitHub for project version control.

Advanced practices such as automated CI/CD, Docker, Kubernetes, and MLflow are planned for future development.

9.3 Most challenging task

One of the most challenging tasks was organizing and processing the large retail transaction dataset.

The dataset contained missing values, invalid transactions, and inconsistent records. These issues affected data quality and analysis.

I addressed this by inspecting the dataset, cleaning invalid records, handling missing values, and performing feature engineering using Python and Pandas.

I verified the solution by checking the processed dataset, date ranges, record counts, and dashboard outputs.

9.4 Future roadmap

The following improvements can be implemented in future versions:

1. Use larger historical datasets for better demand forecasting.
2. Improve forecasting with time-aware validation.
3. Evaluate churn models using Precision, Recall, F1-score, and ROC-AUC.
4. Integrate real-time stock levels and supplier lead times.
5. Implement automated testing.
6. Integrate MLflow for experiment tracking.
7. Implement Docker and Kubernetes deployment.

Conclusion

RetailPulse is an AI-powered retail analytics and demand forecasting platform developed using Python, Pandas, Scikit-learn, and Streamlit.

The project combines retail sales analysis, RFM-based customer segmentation, churn-risk prediction, monthly demand forecasting, and inventory recommendations in a single interactive dashboard.

This project provided practical experience in data science, machine learning, data preprocessing, and dashboard development. It also helped me understand how machine learning techniques can support retail business decision-making

10. Final Submission Checklist

Complete this checklist after verifying the application and adding your own evidence.

☐ All numbers and model metrics have been reproduced from the current project run.

☐ All module-wise screenshot placeholders have been replaced with genuine screenshots.

☐ Project timeline contains actual dates and deliverables.

☐ Personal reflection is written in my own words and reflects my actual work.

☐ Live demo URL is added only if the app is publicly deployed and tested.

☐ GitHub repository URL is correct and accessible.

☐ README, requirements and setup instructions are current.

☐ No credentials, API keys, private customer data or secrets are included.

☐ No unimplemented tools are described as completed.

☐ PDF is A4, 10–18 pages, and within the submission file-size limit.

Project links

GitHub repository: - https://github.com/shekharshelke171-eng/RetailPulse

Live demo (HTTPS): - https://retailpulse-analytics-171.streamlit.app/

Demo video: https: - //drive.google.com/drive/my-drive?dmr=1&ec=wgc-drive-%5Bmodule%5D-goto

Submission date: - November 2026

Prepared for review by Shekhar Shelke | Zidio Development | Data Science & Analytics